IMPROVED LEARNING OF WORD EMBEDDINGS WITH WORD DEFINITIONS AND SEMANTIC INJECTION

Yichi Zhang[1], *Yinpei Dai*[1], *Zhijian Ou*[1†], *Huixin Wang*[2], *Junlan Feng*[2]

[1]Speech Processing and Machine Intelligence (SPMI) Lab, Tsinghua University, China
[2]China Mobile Research Institute

{zhangyic17, dyp16}@mails.tsinghua.edu.cn, ozj@tsinghua.edu.cn, {wanghuixin,fengjunlan}@chinamobile.com

ABSTRACT

Recently, two categories of linguistic knowledge sources, word definitions from monolingual dictionaries and linguistic relations (e.g. synonymy and antonymy), have been leveraged separately to improve the traditional co-occurrence based methods for learning word embeddings. In this paper, we investigate to leverage these two kinds of resources together. Specifically, we propose a new method for word embedding specialization, named Definition Autoencoder with Semantic Injection (DASI). In our experiments, DASI outperforms its single-knowledge-source counterparts on two semantic similarity benchmarks, and the improvements are further justified on a downstream task of dialog state tracking. We also show that DASI is superior over simple combinations of existing methods in incorporating the two knowledge sources.

Index Terms— Word embedding specialization, word definitions, semantic injection, dialog state tracking.

1. INTRODUCTION

Distributed representations of words, also known as word embeddings, have been successfully used in many natural language processing (NLP) tasks [1, 2]. Traditional methods of learning word embeddings over unlabeled textual corpora are mainly based on the distributional hypothesis [3], which states that words occuring in similar contexts tend to be semantically close.

These co-occurrence based methods are good at capturing relatedness between words, but usually lack in capturing similarity and distinguishing similarity from relatedness [4]. For example, "cup" and "coffee" often appear together - they are related but not similar. In contrast, "cheap" and "inexpensive" are similar but seldom co-occur in a sentence. This drawback can hurt the performance of downstream language understanding tasks such as dialog state tracking [5], which aims at tracking users' preference over semantic slots expressed in user utterances, as shown in Table 1.

This weakness of the co-occurrence based methods can be alleviated by introducing linguistic supervisions in the learning process. Two classes of external semantic knowledge sources have been leveraged separately. First, *word definitions* in monolingual dictionaries, which contain semantic descriptions about words, are used to enhance the learning of word embeddings, especially for capturing similarity between words [6, 7, 8]. Second, distributional word vectors can be refined by injecting semantic relations (e.g. synonymy and antonymy) from lexical resources such as WordNet [9] and Paraphrase Database [10], as shown in [11, 12, 13, 14]. We term this process *semantic injection*.

This work is supported by NSFC 61976122, Ministry of Education and China Mobile joint funding MCM20170301. † Corresponding author.

Restaurant price range

Slot value	Synonyms
cheap	cheaper, inexpensive, bargain, ...
moderate	mid-price, affordable, medium, ...
expensive	costly, pricy, dear, ...

Table 1. Examples of different expressions of users' preference over restaurant price ranges. Language understanding models need to classify synonymous expressions into the correct class (slot value) and distinguish antonymous ones such as "cheap" and "expensive".

However, for learning with dictionary definitions or with semantic injection, either alone has its own limitation. For learning with dictionary definitions alone, semantic relations between words are not explicitly enforced in learning word embeddings, since a word's synonyms and antonyms are not always appeared in its definition. For learning with semantic injection alone, although semantic relations are directly injected, the descriptive information contained in dictionary definitions are not exploited. For example, the phrase "charging low prices" in the definition of word "cheap" clearly carries useful information for capturing similarity between words.

In this paper, we propose a new method for word embedding post-processing that specializes in similarity relation, named Definition Autoencoder with Semantic Injection (DASI), which leverages both knowledge sources. The connection and comparison of DASI with existing methods are detailed in Section 2. Two state-of-the-art specialization methods, the ATTRACT-REPEL (A-R) [14] and Consistency Penalized AutoEncoder (CPAE) [8], are used as the baselines of semantic injection and definition modeling respectively. It is found in our experiments that DASI outperforms its single-knowledge-source counterparts in three different vector spaces on two word similarity benchmarks: SimLex-999 [4] and SimVerb-3500 [15]. Furthermore, DASI improves the downstream dialog state tracking (DST) performance over two different DST models.

2. RELATED WORK

Table 2 shows a brief review of existing methods for learning word embeddings, depending on the used external semantic knowledge sources. To the best of our knowledge, this paper represents the first exploration in incorporating both word definitions and semantic injection for word representation learning.

External Resource		Semantic Injection	
		×	✓
Word Definitions	×	word2vec [17] GloVe [18] fastText [19]	retrofitting [11] PARAGRAM [12] counter-fitting [13] ATTRACT-REPEL [14]
	✓	Lexicographic [6] dict2vec [7] CPAE [8]	DASI (ours)

Table 2. Categorization of different word embedding learning methods according to the use of external linguistic resources.

2.1. Learning with Word Definitions

There are two main ways to learn with word definitions. One is to use word definitions as an additional co-occurrence context [6, 7]. The other is to learn definition embeddings which can be used to generate or reconstruct the original word definitions [16, 8]. Our approach is similar to the latter in the spirit of using definition reconstruction to incorporate dictionary knowledge. Specifically, inspired from CPAE [8], DASI consists of encoding the sequence of words in a definition into a vector and trying to reconstruct/decode the definition. A consistency penalty is used to enforce that the input word embeddings it uses as inputs and the definition embeddings it produces as outputs are close to each other, as a result of the inherent recursivity of dictionaries. However, there are two important differences between DASI and CPAE, as will be detailed in Section 3.1 with experimental comparison in Section 5.

2.2. Learning with Semantic Injection

Semantic relations in semantic lexicons such as WordNet [9] and Paraphrase Database (PPDB) [10] have been shown to be useful in fine-tuning and specializing pre-trained word embeddings to better capture word similarity [11, 12, 13, 14]. Specifically, retrofitting [11] brings the vectors of semantically similar words close together. PARAGRAM [12] injects paraphrasing constraints from PPDB to the original skip-gram objective function [17] to fine-tune the word vectors. Counter-fitting [13] and ATTRACT-REPEL [14] leverage both synonymy and antonymy constraints as semantic injection sources, and use a hinge loss to pull synonymy pairs closer and push antonymy pairs away to reach the pre-defined similarity margin.

In DASI, we also use synonymy and antonymy constraints but with a different objective function so that we can naturally combine word definition modeling and semantic injection. In particular, synonyms are attracted by increasing the conditional likelihood of its synonyms for a given word, and antonyms are repelled in the opposite way (decreasing the conditional likelihood). This difference is detailed in Section 3.2 with experimental comparison in Section 5.

3. DEFINITION AUTOENCODER WITH SEMANTIC INJECTION

Denote by e_w the target embedding for word w, which is initially in a pre-trained word vector space (e.g. word2vec [17]). Suppose that we have access to a dictionary consisting of word definitions and a semantic lexicon consisting of synonyms and antonyms pairs.

- Denote by $\mathcal{D}$ the set of the words which have definitions. Let $D_w = \{D_{w,t}, t = 1, \cdots, |D_w|\}$ be the concatenated sequence of words from all the definitions for a word $w \in \mathcal{D}$, since a word may have multiple senses in the dictionary[1]. Denote by $\mathcal{E} \triangleq \{w' | w' \in D_w, w \in \mathcal{D}\}$ the set of words that are used to define words in $\mathcal{D}$.
- Denote by $\mathcal{M}$ the set of the words which have synonyms and/or antonyms. Let S_w and A_w be the set of synonyms and antonyms of word $w \in \mathcal{M}$ respectively.

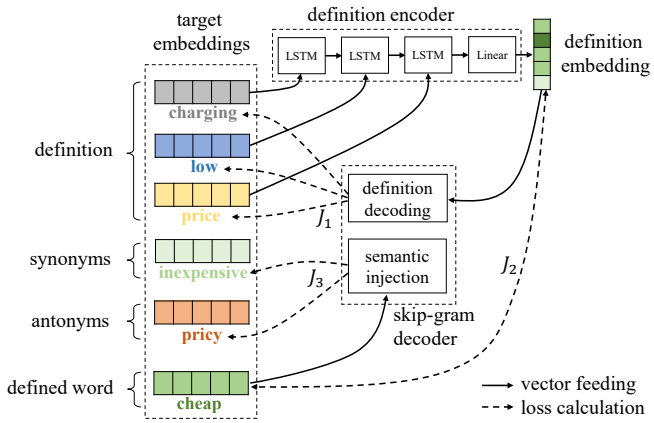

Fig. 1. Overview of the DASI model.

Let $e_{\mathcal{E}}$, $e_{\mathcal{E}\cup\mathcal{D}}$ and $e_{\mathcal{M}}$ denote the embeddings of words in the corresponding set of words respectively. Let $\mathcal{V} \triangleq \mathcal{D} \cup \mathcal{E} \cup \mathcal{M}$ be the total vocabulary. Then we can improve $\{e_w | w \in \mathcal{V}\}$, by leveraging both the word definitions and the synonyms and antonyms, via DASI as shown in Fig.1.

3.1. Definition Modeling

Word definitions are incorporated by using an autoencoder model inspired from CAPE. For $w \in \mathcal{D}$, an LSTM based encoder runs over the word sequence D_w , where the words in D_w are represented by their target embeddings we want to improve. Then we apply a linear transformation to the last hidden state of LSTM to obtain the definition embedding d_w that is of the same dimension as e_w :

$$d_w = f_\theta(D_w) = W \cdot LSTM(D_w) + b$$

where θ denotes the encoder parameters, consisting of the LSTM parameters, and $\{W, b\}$. A skip-gram based decoder, which treats D_w as a bag-of-words, is introduced to define a reconstruction loss J_1:

$$J_1(e_{\mathcal{E}}, \theta, \phi) = -\sum_{w\in\mathcal{D}} \sum_{t=1}^{|D_w|} \log q_\phi(D_{w,t} | d_w)$$

Here $q_\phi(\cdot|\cdot)$ denotes the skip-gram probability, which can be generally defined for word $u \in \mathcal{V}$ appearing in the context of word $w \in \mathcal{D}$:

$$\log q_\phi(u|d_w) = \log \frac{\exp(W_u^{\mathrm{T}} d_w + b_u)}{\sum_{k\in\mathcal{V}} \exp(W_k^{\mathrm{T}} d_w + b_k)} \tag{1}$$

where W_k and b_k denote the weight and bias for word $k \in \mathcal{V}$, and ϕ denotes the decoder parameter $\{W_k, b_k | k \in \mathcal{V}\}$.

To incorporate the definition information compressed in d_w into e_w, we borrow the consistency penalty in [8], which is to make e_w

[1] With abuse of notation, D_w may also represent the set of words in the sequence D_w in this paper. $|D_w|$ denotes the length/size.

and d_w be close to each other :

$$J_2(e_{\mathcal{D}\cup\mathcal{E}}, \theta) = \sum_{w\in\mathcal{D}} dist(e_w, d_w)$$

where $dist$ is a distance measurement and we choose the Euclidean distance in our experiment.

DASI draws inspiration from CPAE, but differs from CPAE in two important ways. First, DASI aims to improve the target embeddings, by feeding them into the LSTM and enforcing reconstruction and consistency to fine-tune the target embeddings. In contrast, CPAE adopts the definition embedding from the LSTM output as the target embedding, after training the LSTM from scratch. Fine-tuning is much faster than learning from scratch, and has the advantage of only making necessary changes to pre-trained vectors. Section 5 further provides experimental validation. Second, apart from used in definition decoding, we propose to use the skip-gram model to do semantic injection in DASI, which naturally integrates two knowledge sources and is found to be superior to other simple combinations of existing methods as shown in our experiments.

3.2. Semantic Injection

We inject the semantic relation constraint by increasing the conditional log-probability of synonyms and decreasing the conditional log-probability of antonyms for $w \in \mathcal{M}$ as follows:

$$J_3(e_{\mathcal{M}}, \phi) = -\sum_{w\in\mathcal{M}} \left(\sum_{s\in S_w} \log q_\phi(s|e_w) - \sum_{a\in A_w} \log q_\phi(a|e_w) \right)$$

Here we propose to reuse the skip-gram model $q_\phi(\cdot|\cdot)$ in Eq. (1) to calculate the probability of synonyms/antonyms appearing in the context of word w, since e_w and d_w are of the same dimension. The semantic injection penalty pulls the defined word embedding e_w close to the embedding of synonyms e_s where $s \in S_w$, and pushes e_w away from the embedding of antonyms e_a where $a \in A_w$. The motivation of reusing the skip-gram decoder for semantic injection is to incorporate word definitions and semantic relations in a consistent manner.

The overall objective function is given by the weighted sum of the three losses described above:

$$J(e_{\mathcal{V}}, \theta, \phi) = J_1(e_{\mathcal{E}}, \theta, \phi) + \alpha J_2(e_{\mathcal{D}\cup\mathcal{E}}, \theta) + \beta J_3(e_{\mathcal{M}}, \phi)$$

where α and β are hyperparamters to control the balance of each loss.

4. EXPERIMENTS

We compare the proposed method with two state-of-the-art models, CPAE [8] and ATTRACT-REPEL (shortened as A-R) [14], using either word definitions or semantic relations. We conduct experiments for both intrinsic and downstream evaluations. Details of training and evaluation settings are described as follows.

4.1. Training Settings

We use word definitions and semantic relations in WordNet [9]. WordNet is a large human-constructed semantic lexicon for English words. It groups words into sets of synonyms called *synsets*, provides their definitions, and records the various semantic relations between synsets. There are 117,597 synsets and 207,016 semantic relation pairs. We use the definitions in WordNet as the resource of dictionary definitions, and the annotated synonymy and antonymy as the resource of semantic injection to train our models. For words that have more than one sense (included in more than one synsets in WordNet), their definitions are the concatenation of definitions in all senses and the synonym set and antonym set are also the union of each in all senses respectively.

Pre-trained Embedding	Specialization Method	Similarity Scores		
		SV-dev	SV-test	SL
word2vec	-	39.20	35.78	44.09
	A-R	59.60	54.61	65.87
	CPAE	44.59	41.73	44.63
	DASI	**63.77**	**60.75**	**67.59**
GloVe	-	26.92	22.20	36.89
	A-R	51.46	44.48	59.88
	CPAE	35.13	28.68	40.20
	DASI	**59.56**	**54.88**	**64.03**
Paragram-SL999	-	52.80	54.21	68.41
	A-R	60.57	60.02	72.99
	CPAE	60.63	58.35	68.12
	DASI	**66.81**	**65.54**	**73.95**

Table 3. Results of different post-processing methods for three kinds of embeddings on SimVerb-3500 (SV) and SimLex-999 (SL). Best results are in bold.

We use three kinds of pre-trained word embeddings in our experiments. The word2vec[2] [17] and GloVe [3] [18] which are distributed word vectors trained on large text corpus, and Paragram-SL999 [4] [12] which is a specialization of word2vec using the Paraphrase Database [10]. All of the embeddings are 300 dimensions.

The implementations of CPAE and ATTREP-REPEL are based on the open-source code available here [5]. We use a batch size of 256 and employ the Adam optimizer for all the methods. For DASI, we set $\alpha = 25, \beta = 1.0$ and use a learning rate of 0.001. We set $\lambda = 32$ and a learning rate of 0.001 for CPAE and $\delta_{sim} = 1.0$, $\delta_{ant} = 0.0$, $\lambda_{reg} = 10^{-9}$ and a learning rate of 0.05 for ATTRACT-REPEL. All the hyperparameters are chosen by grid search. We use the standard development set of SimVerb-3500 [15] for validation check, and training early stops when no improvement is observed within 100 updating steps. The model with the best validation score is used for intrinsic and downstream evaluations. The vocabulary size is 20,000 in our experiments, which includes almost all the words in the word similarity benchmarks and the downstream dialog dataset.

4.2. Evaluation Settings

We conduct the intrinsic evaluation on two word similarity datasets: SimLex-999 [4] and SimVerb-3500 [15], which contain human annotated similarity ratings for 999 and 3500 word pairs respectively. For embedding evaluation, we computes the cosine similarity for each word vector pair and measure the *Spearman correlation* ρ between predicted score ranking and ground truth ranking.

We expect that the improvement of word similarity capturing can benefit downstream tasks such as language understanding, and we choose the dialog state tracking (DST) task for evaluation. DST aims to capture user goals, which are expressed by slot-value pairs

[2] https://code.google.com/archive/p/word2vec/
[3] https://nlp.stanford.edu/projects/glove/
[4] https://cogcomp.org/page/resource_view/106
[5] CPAE: https://github.com/tombosc/cpae; ATTRACT-REPEL: https://github.com/nmrksic/attract-repel

Embedding	NBT		GLAD	
	Joint goal	Request	Joint goal	Request
Paragram-SL999	88.71	97.11	81.94	93.52
+ A-R	86.41	96.94	83.08	**93.88**
+ CPAE	88.96	97.15	81.80	93.76
+ DASI	**89.78**	**97.40**	**84.32**	93.34

Table 4. Joint goal accuracy and request accuracy of two models with different input embeddings on WOZ 2.0. Paragram-SL999 is chosen as the baseline embedding and three post-processing methods are compared. Best results are in bold.

Method	Epoch	SV-dev	SV-test	SL
CPAE (input)	**24.5**	44.59	**44.63**	**41.73**
CPAE (definition)	34.6	**45.56**	41.72	38.76
DASI (input)	**33.2**	**63.77**	**60.75**	**67.59**
DASI (definition)	47.1	57.35	54.17	54.96

Table 5. Comparison of applying input embeddings versus definition embeddings in evaluating definition autoencoder based models. Epoch denotes the average epoch number when training early stops. Best results are in bold.

such as *food=India* or *area=North*, given user utterances. The performance are measured by *joint goal accuracy* and *request accuracy*, which represent the proportion of turns with all the slot-value pairs correctly classified and all the requests correctly answered respectively.

The evaluation is based on WOZ 2.0 dataset [20], which consists of 600/200/400 dialogs in training/development/testing sets respectively in the restaurant domain. The system is required to track user's preference on price range, restaurant location and food type, where many variants (southern for south, cheaper for cheap etc.) and synonyms (costly for expensive etc.) are used in user expressions. Two baseline DST models are used, the Neural Belief Tracker (NBT) [20] and the Global-Locally Self-Attentive Dialogue State Tracker (GLAD) [21]. Both of them use fixed word embeddings as input which allows a fair comparison among different input embeddings. We train NBT and GLAD following their default settings[6].

5. RESULTS AND ANALYSIS

We report the Spearman's correlation coefficient $\rho \times 100$ on SimVerb-3500 and SimLex-999 datasets as metric of word similarity capturing. A-R in the tables denotes the shortcut of ATTRACT-REPEL. All the scores in each table are the average of 5 runs with different random seeds. Results are organized to show three conclusions:

DASI outperforms its single-knowledge-source counterparts under various cases. In Table 3, it can be seen that both pre-trained distributed embeddings (word2vec and GloVe) and specialized embeddings (Paragram-SL999) are improved through injecting external linguistic knowledge. We find that DASI significantly outperforms CPAE and ATTRACT-REPEL in all cases, which shows that word definitions and semantic relations can work together and make a further improvement of embedding qualities. For downstream evaluation, we only report the results of Paragram-SL999 with different post-processing methods since they are better than results of word2vec and GloVe. The significant increase of joint goal accuracy indicates that better capturing of word similarities benefits the tracking of user goals, as shown in Table 4. The consistent improvement of DASI over multiple pre-trained embeddings and under different DST models suggest that DASI generalizes well.

[6]GLAD: `https://github.com/salesforce/glad`; NBT: `https://github.com/nmrksic/neural-belief-tracker`

Method	SV-dev	SV-test	SL
original	39.20	35.78	44.09
^{a}A-R $\rightarrow$ CPAE	59.31	54.45	65.74
^{b}CPAE $\rightarrow$ A-R	59.67	56.53	64.17
^{c}CPAE + A-R loss	43.01	38.90	47.14
^{d}DASI (w/o weight tying)	62.73	**60.79**	66.55
DASI	**63.77**^{abcd}	60.75^{abc}	**67.59**^{abcd}

Table 6. Comparison of different methods for fusing word definitions and semantic relations. A-R$\rightarrow$CPAE denotes post-processing by ATTRECT-REPEL first and CPAE later, while A-R$\rightarrow$CPAE the inverse. CPAE+A-R loss denotes the model that replaces the J_3 loss in DASI with the pair-wise hinge loss in ATTRACT-REPEL. DASI without weight tying uses independent definition decoder and semantic injection model. Superscripts in the last line denote statistical significance (p<0.01) over the four baselines a, b, c and d.

The effect of adopting different embeddings in incorporating definition knowledge shows the superiority of DASI over CPAE. Table 5 shows that using the input embeddings (as proposed in DASI) rather than the output definition embeddings from the definition autoencoder (as proposed in CPAE) has better similarity scores and faster convergence speed. This indicates that fine-tuning on the original pre-trained vectors is more efficient than learning new vectors by adjusting the LSTM encoder parameters from scratch, since the model can make only necessary changes to the pre-trained vectors.

DASI outperforms simple combinations of existing methods in fusing both sources of external knowledge. We compare DASI with several baseline models incorporating knowledge from both word definitions and semantic relations. As shown in Table 6, simultaneously utilizing multiple external knowledge resources within a single training process outperforms successive utilizations. For DASI, using the skip-gram model obtains a significant better result than directly adding the loss of ATTRACT-REPEL to CPAE, presumably because of the consistent manner in DASI as discussed in Section 3.2. Sharing the parameters in the definition decoder and semantic injection model for DASI results in better similarity scores and a more compact model.

6. CONCLUSION AND FUTURE WORK

This work represents a first exploration of incorporating two kinds of external linguistic knowledge resources, the word definitions and semantic relations together to do specialization of word vectors. We develop a new DASI method, which shows significant improvement over its single-knowledge-source counterparts on both intrinsic and downstream evaluations, across various pre-trained embeddings and DST models. We also analyze and compare different fusion approaches to show that DASI is a better solution for knowledge fusion.

There are some interesting future works. First, since only the words contained in dictionary or with semantic relations are specialized in DASI, how to leverage recent progress in learning a global specialization function [22, 23] to overcome this limitation is interesting. Second, while DASI makes use of two external linguistic knowledge resources, other resources such as the example sentences in dictionaries which might be helpful as well are worthwhile to exploration.

7. REFERENCES

[1] Joseph Turian, Lev Ratinov, and Yoshua Bengio, "Word representations: a simple and general method for semi-supervised learning," in *Proceedings of the 48th annual meeting of the association for computational linguistics*. Association for Computational Linguistics, 2010, pp. 384–394.

[2] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel Kuksa, "Natural language processing (almost) from scratch," *Journal of machine learning research*, vol. 12, no. Aug, pp. 2493–2537, 2011.

[3] Zellig S Harris, "Distributional structure," *Word*, vol. 10, no. 2-3, pp. 146–162, 1954.

[4] Felix Hill, Roi Reichart, and Anna Korhonen, "Simlex-999: Evaluating semantic models with (genuine) similarity estimation," *Computational Linguistics*, vol. 41, no. 4, pp. 665–695, 2015.

[5] Jason Williams, Antoine Raux, Deepak Ramachandran, and Alan Black, "The dialog state tracking challenge," in *Proceedings of the SIGDIAL 2013 Conference*, 2013, pp. 404–413.

[6] Tong Wang, Abdelrahman Mohamed, and Graeme Hirst, "Learning lexical embeddings with syntactic and lexicographic knowledge," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, 2015, vol. 2, pp. 458–463.

[7] Julien Tissier, Christophe Gravier, and Amaury Habrard, "Dict2vec: Learning word embeddings using lexical dictionaries," in *Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)*, 2017, pp. 254–263.

[8] Tom Bosc and Pascal Vincent, "Auto-encoding dictionary definitions into consistent word embeddings," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 1522–1532.

[9] George A Miller, "Wordnet: a lexical database for english," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[10] Juri Ganitkevitch, Benjamin Van Durme, and Chris Callison-Burch, "Ppdb: The paraphrase database," in *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2013, pp. 758–764.

[11] Manaal Faruqui, Jesse Dodge, Sujay Kumar Jauhar, Chris Dyer, Eduard Hovy, and Noah A Smith, "Retrofitting word vectors to semantic lexicons," in *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2015, pp. 1606–1615.

[12] John Wieting, Mohit Bansal, Kevin Gimpel, and Karen Livescu, "From paraphrase database to compositional paraphrase model and back," *Transactions of the Association for Computational Linguistics*, vol. 3, pp. 345–358, 2015.

[13] Nikola Mrkšic, Diarmuid OSéaghdha, Blaise Thomson, Milica Gašic, Lina Rojas-Barahona, Pei-Hao Su, David Vandyke, Tsung-Hsien Wen, and Steve Young, "Counter-fitting word vectors to linguistic constraints," in *Proceedings of NAACL-HLT*, 2016, pp. 142–148.

[14] Nikola Mrkšić, Ivan Vulić, Diarmuid Ó Séaghdha, Ira Leviant, Roi Reichart, Milica Gašić, Anna Korhonen, and Steve Young, "Semantic specialization of distributional word vector spaces using monolingual and cross-lingual constraints," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 309–324, 2017.

[15] Daniela Gerz, Ivan Vulić, Felix Hill, Roi Reichart, and Anna Korhonen, "Simverb-3500: A large-scale evaluation set of verb similarity," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2173–2182.

[16] Thanapon Noraset, Chen Liang, Larry Birnbaum, and Doug Downey, "Definition modeling: Learning to define word embeddings in natural language," in *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[17] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in neural information processing systems*, 2013, pp. 3111–3119.

[18] Jeffrey Pennington, Richard Socher, and Christopher Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.

[19] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[20] Nikola Mrkšić, Diarmuid Ó Séaghdha, Tsung-Hsien Wen, Blaise Thomson, and Steve Young, "Neural belief tracker: Data-driven dialogue state tracking," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1777–1788.

[21] Victor Zhong, Caiming Xiong, and Richard Socher, "Global-locally self-attentive dialogue state tracker," *arXiv preprint arXiv:1805.09655*, 2018.

[22] Goran Glavaš and Ivan Vulić, "Explicit retrofitting of distributional word vectors," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 34–45.

[23] Edoardo Maria Ponti, Ivan Vulić, Goran Glavaš, Nikola Mrkšić, and Anna Korhonen, "Adversarial propagation and zero-shot cross-lingual transfer of word vector specialization," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 282–293.

[24] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer, "Deep contextualized word representations," *arXiv preprint arXiv:1802.05365*, 2018.

[25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.